

001 Quick Start Guide

Einleitung: Was ist Agate Test Studio?

Frage: "Willst du Tests schnell und effizient automatisieren?"

Agate Test Studio ist ein leichtgewichtiges Testautomatisierungs-Framework, das weder Programmiercode noch teure Software-Lizenzen erfordert. Es hilft dir, komplexe Systemabläufe (REST, SOAP, Datenbanken, OpenShift) ohne aufwendige Installationen direkt zu validieren. Agate bringt Ordnung in deine Testlandschaft.

Systemvoraussetzungen (Was du brauchst)

Damit Agate einwandfrei funktioniert, stelle sicher, dass folgende Werkzeuge auf deinem Rechner bereitstehen:

- **Java Runtime Environment (JRE/JDK) 17+:** Das ist die Grundvoraussetzung, um das Agate-Backend auszuführen.
- **OC-Client (oc.exe):** Wenn deine Tests mit OpenShift-Umgebungen interagieren sollen, muss der OpenShift-Client installiert und im System-PATH verfügbar sein.
- **SQLPlus:** Wenn du DB-Engine -Tests durchführst, wird SQLPlus benötigt, um die Datenbankverbindungen aufzubauen.

Schritt 1: Die erste Berührung (Erste Schritte)

Frage: "Kann ich sofort sehen, wie Agate funktioniert?"

Ja! Agate Test Studio ist flexibel. Du startest deine Tests direkt über die `startTests.bat` in deiner Windows-Konsole. Dabei kannst du genau steuern, **wer** du bist, **wo** du testest und **was** genau ausgeführt werden soll.

1. Der Start: So rufst du `startTests.bat` auf

Das Skript folgt einem einfachen Schema:

```
1 startTests.bat [USER] [INSTANCE] [APP] [TEST_SUITE] [TEST_CASE] [PRIORIT
```

Hier sind die gängigsten Szenarien, wie du deine Tests starten kannst:

- **Den kompletten Testlauf für eine App starten:**

```
1 | startTests.bat Tester1 DEMOS demo
```

(Führt alle verfügbaren Tests in der App "demo" aus.)

- **Nur einen spezifischen Test-Suite (Datei) ausführen:**

```
1 | startTests.bat Tester1 DEMOS demo 001_windows_cmd_basic_test
```

(Führt nur die Tests in dieser einen Datei aus.)

- **Einen ganz bestimmten Testfall (Test Case) isolieren:**

```
1 | startTests.bat Tester1 DEMOS demo 001_windows_cmd_basic_test TC_CMD_02
```

(Hilfreich, wenn du an einem einzigen Test feilst.)

- **Nach Priorität filtern (z.B. nur kritische Tests):**

```
1 | startTests.bat Milenko DEMOS demo 001_windows_cmd_basic_test "" HIGH
```

(Hier setzen wir für den Test Case ein Leer-String "" ein, um nur nach Priorität HIGH zu filtern.)

Wichtig: Achte darauf, dass du in der Konsole immer im richtigen Projektordner bist, damit das Skript die `target/`-Datei und deine Konfigurationen findet.

2. Die Analyse: Dein HTML-Report

Während die Konsole dir den Live-Status zeigt, ist der **HTML-Report** dein wichtigstes Werkzeug für die Analyse.

- **Warum ein Report?** Die Konsole ist nur für den Moment. Der HTML-Report hingegen speichert deine Testergebnisse dauerhaft, strukturiert und übersichtlich.
- **Wo finde ich ihn?** Nach jedem Lauf wird automatisch ein Report im Ordner `reports/` erstellt.
 - `Latest_Report.html` : Immer der aktuellste Lauf.
 - Zeitlich gestempelte Reports: Historische Ergebnisse für spätere Vergleiche.

Schritt 2: Die Umgebung verstehen (Env-Setup)

Frage: "Wo arbeite ich und wie konfiguriere ich mein System?"

Um mit Agate Test Studio zu arbeiten, musst du dem System sagen, in welcher Umgebung du dich befindest. Das Setup ist in zwei Dateien unterteilt, die sich im `env/`-Ordner befinden:

1. Die `env/env.conf` (Die Welt deiner Instanzen)

Hier definierst du die "technischen Eckdaten" deiner Zielumgebungen. Jede Instanz beginnt mit einem eindeutigen Namen (z.B. `DEMOS`).

- **Wie füge ich eine neue Umgebung hinzu?** Kopiere einfach einen bestehenden Block und passe den Namen vor dem Punkt an.
- **Was ist wichtig?**
 - Jede Zeile folgt dem Muster: `InstanzName.Parameter=Wert`.
 - Hier legst du globale Werte fest, die für die gesamte Instanz gelten (z.B. IP-Adressen für Services oder Pfade zu Log-Check-Tools).

Hinweis für Profis: Manche Parameter (wie für den Datenbank-Zugriff oder OpenShift-Login) haben feste Namen, die vom Code erwartet werden. Diese werden wir in einem späteren Kapitel im Detail besprechen, wenn wir den SQL- und OC-Engine erklären. Für den Start konzentriere dich erst einmal auf das allgemeine Setup.

2. Die `env/users.conf` (Deine individuelle Arbeitsweise)

Während die `env.conf` das "Wo" definiert, legt die `users.conf` fest, "wer" mit welchen Daten arbeitet. Hier kannst du spezifische Variablen pro Benutzer innerhalb einer Instanz hinterlegen.

- **Wann brauche ich das?** Wenn ein Testfall spezifische Daten eines Benutzers benötigt (z.B. eine Seriennummer einer Karte oder eine spezifische IP-Adresse), wird dies hier definiert.
- **Wie sieht das aus?** `InstanzName.BenutzerName.VariablenName=Wert` *Beispiel:*
`DEMOS.Tester1.seriennummer=99999090`

Schritt 3: Dein eigener Bereich (Application Setup)

Frage: "Wie organisiere ich meine eigenen Tests?"

In Agate Test Studio ist eine "Applikation Unter Test" keine komplizierte Installation. Eine Applikation ist einfach ein **Ordner** unterhalb von `data/`, in dem alle deine Test-Suites, Konfigurationen und Daten zusammengefasst sind.

1. Die Struktur verstehen

Wenn du in das Verzeichnis `data/` schaust, siehst du bereits vorhandene Applikationen wie `demo`, `foo` oder `boo`. Jede dieser Applikationen ist ein eigenständiger Bereich für bestimmte Test-Themen oder Projekte.

2. Deine erste Applikation erstellen

Das ist der schnellste Weg, um zu starten. Du musst nichts im Code ändern oder irgendetwas registrieren – Agate erkennt neue Ordner automatisch.

- **Schritt-für-Schritt:**

- a. Gehe in den Ordner `data/`.
- b. Erstelle einen neuen Ordner mit dem Namen deiner Applikation, zum Beispiel: `MeinErstesProjekt`.
- c. Das war's! Du hast jetzt eine neue Applikation angelegt.

3. Was gehört in den App-Ordner?

Damit Agate deine Tests sauber verarbeiten kann, empfiehlt es sich, die Struktur beizubehalten, die du bereits bei den anderen Applikationen siehst:

- **Hauptordner:** Hier liegen direkt deine `.yaml` Test-Suite Dateien (z.B. `001_mein_test.yaml`).

- **Unterordner (optional):**

- `modules/` : Für wiederverwendbare Anfragen oder API-Definitionen.
- `reusable/` : Für allgemeine Test-Bausteine, die du in verschiedenen Tests brauchst (z.B. "Login").
- `template/` : Für Vorlagen oder Testdaten-Dateien (CSV, XML, etc.).

Tipp: Wenn du eine neue Applikation anlegst, schau dir einfach den Ordner `data/demo` an. Dort siehst du eine perfekte Vorlage, wie du deine Dateien ordnen kannst. Kopiere diese Struktur einfach in deinen neuen Ordner, und du bist sofort startklar.

Schritt 4: Dein erster Testfall (YAML-Struktur)

Frage: "Wie schreibe ich meinen ersten Test?"

Ein Test in Agate Test Studio besteht aus einer YAML-Datei. Um zu verstehen, wie man diese aufbaut, gehen wir schrittweise vor. Wir arbeiten hier in deinem neuen Ordner `MeinErstesProjekt`.

1. Die Basis: `hello_world_01.yaml`

Jeder Test beginnt mit dem `testCases` -Block. Auch ein leerer Testfall ist ein valider Startpunkt.

```
1 | YAML
```

```
1 | # Hello World 1 Demo
2 | testCases:
```

Ergebnis: Wenn du jetzt versuchst, den Test zu starten: `startTests.bat Milenko ECS_SYST_AUT1 MeinErstesProjekt hello_world_01` Agate erkennt die Datei, findet aber noch keine auszuführenden Fälle.

```

1 =====
2           Starting Agate Test Suite via Windows CMD
3 =====
4
5 [INFO] No test cases found matching the provided combination INSTANCE (E
6
7 =====
8           Execution finished.
9 =====

```

2. Ein leerer Testfall: `hello_world_02.yaml`

Jetzt definieren wir einen Testfall. Er ist noch ohne Schritte ("Steps"), aber das Grundgerüst steht:

```

1 | YAML
2
3 | testCases:
4 |   - id: TC Hello World 2
5 |     description: Hello World 2
6 |     stage: "*"
7 |     priority: HIGH
8 |     steps:
9 |

```

Lerneffekt: Du siehst im Report nun ein `[PASSED]`, weil das Gerüst korrekt ist und Agate weiß, was es tun soll (nichts!).

```

1 startTests.bat Milenko ECS_SYST_AUT1 MeinErstesProjekt hello_world_02
2 =====
3           Starting Agate Test Suite via Windows CMD
4 =====
5
6 =====
7 TEST CASE: TC Hello World 2
8 DESC      : Hello World 2
9 STAGE     : * | PRI0: HIGH
10 -----
11 VARIABLES:
12 -----
13
14 -----
15 TEST RESULT: TC Hello World 2 [PASSED]
16 Time: 0,23s | Steps: 0
17 -----
18
19
20 Finalizing suite and cleaning up resources...
21 =====
22           FINAL EXECUTION SUMMARY
23 =====
24 Total Test Cases : 1
25 Passed           : 1
26 Failed           : 0

```

```

27 Success Rate      : 100,0%
28 Total Time       : 0,36s
29 =====
30 OVERALL STATUS    : [SUCCESS]
31 =====
32
33 >>> Archive HTML report generated at: reports/ECS_SYST_AUT1/meinerstesp
34 >>> Shortcut 'Latest' report updated at: reports/ECS_SYST_AUT1/meinerst
35
36 =====
37 Execution finished.
38 =====

```

3. Die Logik: `hello_world_03.yaml` (CMD Engine)

Hier führen wir **Test-Steps** ein. Jeder Step besteht aus:

- **type** : Welcher **Engine** arbeitet? (z.B. **CMD** für Windows-Befehle).
- **op** : Was soll der **Engine** tun? (**EXEC** = Ausführen, **BUFFER** = Speichern, **ASSERT** = Prüfen).
- **response** : Ein Name, unter dem das Ergebnis gespeichert wird, um es in späteren Schritten wiederzuverwenden.

Beispiel:

```

1 | YAML
1 |
2 | steps:
3 |   - type: CMD
4 |     op: EXEC
5 |     command: "java -version"
6 |     response: java_out
7 |   - type: CMD
8 |     op: ASSERT
9 |     action: EXITCODE
10 |    expected: 0

```

4. Variablen nutzen: `hello_world_04.yaml`

Damit deine Tests wartbar werden, kannst du Variablen definieren. So musst du Befehle nicht mehrfach schreiben.

```

1 | YAML
1 |
2 | testCases:
3 |   - id: TC Hello World 1
4 |     variables:
5 |       command: "java -version"
6 |     steps:
7 |       - type: CMD
8 |         op: EXEC
9 |         command: "{B[command]}"

```

Lerneffekt: `{B[variable_name]}` ist die Syntax, um auf deine definierten Variablen zuzugreifen.

Warum ist das wichtig?

- **Modularität:** Durch `type` und `op` kannst du Agate später auf alles "loslassen" (SQL, REST, SOAP), ohne die Logik des Testfalls zu ändern.
- **Datenfluss:** Der `response` -Parameter sorgt dafür, dass Daten von einem Schritt zum nächsten fließen.
- **Variablen:** Halten deinen Test sauber und erlauben es dir, Konfigurationen vom eigentlichen Testablauf zu trennen.

Tip: Wenn du einen Fehler im Test bekommst, wirf immer einen Blick in den `Latest_Report.html`. Agate zeigt dir dort genau an, welcher `type` und `op` in welchem Schritt gescheitert ist.

Beispiel Console:

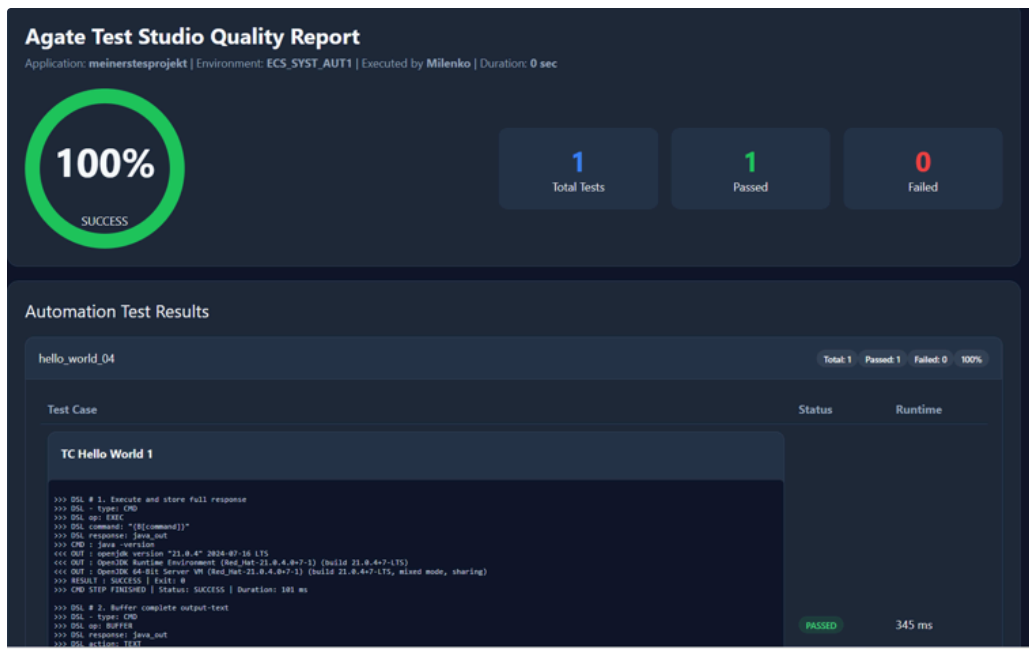
```
1 startTests.bat Milenko ECS_SYST_AUT1 MeinErstesProjekt hello_world_04
2 =====
3           Starting Agate Test Suite via Windows CMD
4 =====
5
6 =====
7 TEST CASE: TC Hello World 1
8 DESC      : Hello World 1
9 STAGE     : * | PRI0: HIGH
10 -----
11 VARIABLES:
12   command      = java -version
13 -----
14
15 >>> DSL      # 1. Execute and store full response
16 >>> DSL      - type: CMD
17 >>> DSL      op: EXEC
18 >>> DSL      command: "{B[command]}"
19 >>> DSL      response: java_out
20 >>> CMD      : java -version
21 <<< OUT      : openjdk version "21.0.4" 2024-07-16 LTS
22 <<< OUT      : OpenJDK Runtime Environment (Red_Hat-21.0.4.0+7-1)
23 <<< OUT      : OpenJDK 64-Bit Server VM (Red_Hat-21.0.4.0+7-1) (bu
24 >>> RESULT   : SUCCESS | Exit: 0
25 >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 99 ms
26
27 >>> DSL      # 2. Buffer complete output-text
28 >>> DSL      - type: CMD
29 >>> DSL      op: BUFFER
30 >>> DSL      response: java_out
31 >>> DSL      action: TEXT
32 >>> DSL      name: var_version_only
33 >>> BUFFER    : [java_out] | Save Full Output to [var_version_only]
34 <<< OUT      : openjdk version "21.0.4" 2024-07-16 LTS
35 <<< OUT      : OpenJDK Runtime Environment (Red_Hat-21.0.4.0+7-1)
36 <<< OUT      : OpenJDK 64-Bit Server VM (Red_Hat-21.0.4.0+7-1) (bu
37 >>> RESULT   : SUCCESS | Buffered
```

```

38     >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 0 ms
39
40 >>> DSL      # 3. Assert Exit Code
41 >>> DSL      - type: CMD
42 >>> DSL      op: ASSERT
43 >>> DSL      response: java_out
44 >>> DSL      action: EXITCODE
45 >>> DSL      expected: 0
46     >>> ASSERT      : EXITCODE | Expected: [0]
47     >>> RESULT      : SUCCESS | Exit: 0
48     >>> CMD STEP FINISHED | Status: SUCCESS | Duration: 0 ms
49
50 -----
51 TEST RESULT: TC Hello World 1                                [PASSED]
52 Time: 0,34s | Steps: 3
53 -----
54
55
56 Finalizing suite and cleaning up resources...
57 =====
58                               FINAL EXECUTION SUMMARY
59 =====
60   Total Test Cases : 1
61   Passed           : 1
62   Failed           : 0
63   Success Rate     : 100,0%
64   Total Time       : 0,47s
65 =====
66   OVERALL STATUS   : [SUCCESS]
67 =====
68
69 >>> Archive HTML report generated at: reports/ECS_SYST_AUT1/meinerstesp
70 >>> Shortcut 'Latest' report updated at: reports/ECS_SYST_AUT1/meinerst
71
72 =====
73                               Execution finished.
74 =====

```

Beispiel Report:



Schritt 6: Dein erster "Hello World" Test (Zusammenfassung)

Frage: "Kann ich meinen eigenen Test jetzt ausführen?"

Du hast jetzt alle Werkzeuge in der Hand. Hier ist der "Fast-Track", um alles zu verifizieren:

- Projekt anlegen:** Erstelle einen neuen Ordner in `data/` (z.B. `MeinErstesProjekt`).
- Datei erstellen:** Lege eine `hello_world_05.yaml` Datei an.
- Test definieren:** Nutze die Struktur aus Schritt 4 (id, description, variables, steps).
- Ausführen:** Starte den Test über die Konsole: `startTests.bat Milenko ECS_SYST_AUT1 MeinErstesProjekt hello_world_05`
- Analysieren:** Öffne `reports/.../Latest_Report.html` und klicke auf den Testnamen, um deine Schritte "aufzuklappen".

7. Steuerung des Testlaufs: Stage und Priority (Header-Filterung)

Frage: "Wie steuert Agate, welche Testfälle in welcher Umgebung ausgeführt werden?"

Im Header jedes Testfalls (`TestCase`) befinden sich zwei Steuerungsfelder: `stage` und `priority`. Das Agate-Backend, um vor jedem Testlauf anhand der System-Properties (`-DSTAGE` und `-DPRI0`) zu entscheiden prüft, ob ein Testfall ausgeführt oder übersprungen wird.

```

1 | - id: F00 anfrage
2 |   description: F00 anfrage
3 |   stage: "DEMOS"
4 |   priority: HIGH
5 |   variables:

```

2.1.1. Das `stage` -Feld (Umgebungs-Filterung)

Das `stage` -Feld bestimmt, für welche Zielinstanz oder Testumgebung (z. B. Entwicklung, Demo, Produktion) der Testfall geschrieben wurde. Beim Starten des Test-Skripts wird dieser Wert mit dem Parameter `[INSTANCE]` (System-Property `STAGE`) verglichen.

Verhaltensmatrix für `stage` (Case-Insensitive):

Zustand im YAML (<code>stage</code>)	Aufruf im Befehl (<code>STAGE</code>)	Verhalten der Engine & Begründung
* (Wildcard)	<i>Beliebige Instanz</i> (z.B. <code>DEMOS</code>)	Wird ausgeführt. Der Asterisk <code>*</code> ist ein Jokerzeichen und bedeutet, dass der Testfall absolut umgebungsunabhängig ist.
Nicht definiert (<code>null</code>)	<i>Beliebige Instanz</i>	Wird übersprungen (FALSE). Wenn im YAML kein <code>stage</code> -Wert definiert ist (<code>tc.getStage() == null</code>), bricht der Filter für diesen Testfall ab, da keine Zuweisung existiert.
Leerer String (<code>" "</code>)	<i>Beliebige Instanz</i>	Wird übersprungen (FALSE). Ein leeres Feld matcht nicht den übergebenen Instanznamen.
Definiert (z.B. <code>DEMOS</code>)	<i>Nicht übergeben / Leer</i>	Wird ausgeführt (TRUE). Wenn der Benutzer beim Start keine spezifische

		Instanz erzwingt (<code>stageParam == null</code>), lässt Agate standardmäßig alle Testfälle durch.
Definiert (z.B. <code>DEMOS</code>)	<code>DEMOS</code> (oder <code>demos</code>)	Wird ausgeführt (TRUE). Der String- Vergleich matcht exakt (Groß-/Kleinschreibung wird ignoriert).
Komma-getrennt (<code>foo, DEMOS, boo</code>)	<code>DEMOS</code>	Wird übersprungen (FALSE). <i>Achtung:</i> Der aktuelle Code prüft auf exakte String- Gleichheit (<code>equalsIgnoreCase</code>) . Eine Listen- Erkennung oder ein <code>.contains()</code> ist im Code nicht implementiert. Der Testfall würde nur starten, wenn die System-Property exakt als <code>foo, demos,</code> <code>boo</code> übergeben wird.

2.1.2. Das `priority` -Feld (Wichtigkeit & Testtiefe)

Mit dem Feld `priority` steuerst du die Testtiefe. Agate nutzt hierbei eine hierarchische Filterung. Das bedeutet: **Ein höher priorisierter Testfall wird immer dann ausgeführt, wenn eine niedrigere oder gleiche Priorität angefordert wird.**

Die interne Hierarchie ist wie folgt definiert:

“LOW” < “MEDIUM” < “HIGH” < “CRITICAL”

Verhaltensmatrix für `priority` :

Zustand im YAML (priority)	Aufruf im Befehl (PRIO)	Verhalten der Engine & Begründung
<i>Beliebiger Wert</i>	Nicht definiert / Leer	Wird ausgeführt (TRUE). Wenn beim Starten kein Filter gesetzt wird, führt Agate ungeachtet der Priorität jeden Testfall aus.
Nicht definiert (oder ungültig)	HIGH	Wird übersprungen (FALSE). Wenn im YAML keine Priorität definiert ist, fällt Agate intern automatisch auf die Stufe LOW zurück. Da $LOW < HIGH$, wird die Bedingung nicht erfüllt.
LOW	HIGH	Wird übersprungen (FALSE). Das Test-Level (LOW) ist kleiner als das erforderliche Level (HIGH).
CRITICAL	HIGH	Wird ausgeführt (TRUE). Das Test-Level (CRITICAL) ist größer als das erforderliche Level (HIGH) .
HIGH	HIGH	Wird ausgeführt (TRUE). Exakter Match ($3 = 3$).
HIGH	<i>Ungültiger String</i> (z.B. F00)	Wird ausgeführt (TRUE). Wenn der Benutzer einen

		ungültigen Prioritäts-Filter übergibt, den Agate nicht kennt, wird der Filter ignoriert und der Testfall sicherheitshalber ausgeführt.
--	--	--

8. Ausblick: Was kommt als nächstes?

Ausblick: Die Zukunft von Agate Test Studio

Agate Test Studio ist ein wachsendes Ökosystem. Damit du dein Wissen gezielt aufbauen kannst, habe ich den Lernpfad in klare Module unterteilt:

Der Lernpfad für Agate-Profis

- **Quick Start Guid (Dieser Guide):** Du hast das Fundament – das Agate Backend, Konfiguration und den ersten eigenen Test – erfolgreich gemeistert.
- **Advanced User Tutorial (In Arbeit):** Hier tauchen wir tief in die einzelnen Engines ein. Wir analysieren im Detail, wie `CMD`, `SQL`, `OC` (OpenShift), `REST` und `SOAP` im Agate-Backend funktionieren.
- **Expert User Tutorial (In Arbeit):** Hier werden wir das volle Potenzial ausschöpfen. Wir konzentrieren uns auf das **DSL-Template-Konzept** und **Data-Driven Testing (DDT)**. Du lernst, wie du mit einer einzigen Vorlage (YAML) und einem CSV-Datensatz tausende Testvariationen automatisiert ausführst.
- **TOSCA Migration Concept (In Arbeit):** Für alle, die von Tosca umsteigen möchten – hier zeigen wir das Konzept, wie du Tosca-Testsets effizient in das Agate DSL-Format migrierst.

Status der "Next Generation" Features

Neben dem stabilen Core-Backend entwickeln sich parallel diese visionären Bereiche:

- **Agate Client & AI-Integration:** Die Client-Server-Architektur mit integriertem AI-Chatbot (Ollama) steht bereit, um Test-Engineering durch KI-Unterstützung zu revolutionieren.
- **Playwright GUI-Engine:** Das nächste Level für Web-Tests mit automatisiertem Site-Capture.

Ein Wort zum aktuellen Status: Agate Test Studio ist ein leidenschaftliches Open-Source-Projekt, das ich in meiner Freizeit entwickle.

Du hast Interesse, an der Zukunft mitzubauen oder Beta-Feedback zu geben? Ich freue mich auf den Austausch!